

AI Homework Report

Sudoku Solving: A* Search vs SAT Encoding on Norvig Benchmarks

First Name Last Name

January 12, 2026

Abstract

This report studies the Sudoku (9×9) puzzle as a benchmark problem for comparing a search-based technique and a declarative constraint-based technique. We implement A* search from scratch under standard constraints (*duplicate elimination* and *no reopening*) and compare it to a SAT-based solver obtained by encoding Sudoku as a CNF formula and calling a modern SAT solver via PySAT. Experiments are conducted on Norvig’s benchmark sets (**easy50**, **top95**, **hardest**). Results show that A* can be effective on easier instances when guided by MRV successor generation and a simple admissible heuristic, but its performance varies significantly on heterogeneous hard sets. In contrast, SAT solving remains stable and fast across all benchmarks.

1 Introduction

Sudoku is a constraint satisfaction problem (CSP) where digits 1–9 must be assigned to a 9×9 grid respecting row/column/box uniqueness constraints. The assignment requires implementing:

- a **search-based** technique (A*), where solution quality and performance depend on branching factor, heuristic guidance, and memory usage;
- an **alternative** technique (SAT), where the problem is reduced to satisfiability in CNF and solved by a dedicated solver.

The goal is not only to solve instances but to provide an engineering analysis of algorithmic choices and performance drivers. In particular, we analyze: (i) the impact of a simple admissible heuristic in A*, (ii) the interaction between runtime and search effort (expanded nodes), and (iii) the robustness of SAT with respect to benchmark difficulty.

Reproducibility. The codebase includes scripts to run all experiments and generate plots from a single CSV file. All results presented in this report are computed using these scripts on the benchmark files listed in Section 5.

2 Problem Definition and Modeling

2.1 Sudoku constraints

Let $G_{r,c} \in \{1, \dots, 9\}$ denote the digit assigned to row r and column c , with $r, c \in \{1, \dots, 9\}$. A valid solution satisfies:

1. **Row constraints:** $\forall r, \{G_{r,1}, \dots, G_{r,9}\} = \{1, \dots, 9\}$,
2. **Column constraints:** $\forall c, \{G_{1,c}, \dots, G_{9,c}\} = \{1, \dots, 9\}$,
3. **Box constraints:** each 3×3 subgrid contains digits 1–9 exactly once.

2.2 State-space formulation (A*)

We formulate Sudoku as a shortest-path problem:

- **State s :** a partial assignment over 81 cells. Internally it is represented as a length-81 vector with values in $\{0, \dots, 9\}$ where 0 denotes an empty cell.
- **Initial state s_0 :** the puzzle clues.
- **Actions:** choose an empty cell and assign a legal digit.
- **Transition:** one additional cell becomes assigned.
- **Step cost:** unit cost, hence $g(s)$ equals the number of assignments made beyond the clues.
- **Goal test:** no empty cells and all constraints satisfied.

2.3 Successor generation with MRV

A key engineering choice is reducing branching factor. For each state, successors are generated using *Minimum Remaining Values* (MRV): select the empty cell with the smallest number of legal values and branch only on those values. This is not used as an A* heuristic; it is part of the *problem dynamics* (successor function) and primarily affects the branching factor rather than admissibility.

3 A* Search Implementation

3.1 Algorithm

A* uses the evaluation function $f(n) = g(n) + h(n)$, where g is the path cost and h is a heuristic estimate of remaining cost. The implementation satisfies the assignment constraints:

- **Duplicate elimination:** a map $g_{\text{best}}[s]$ stores the best discovered cost for each state.
- **No reopening:** once a state is expanded (inserted into `closed`), it is not expanded again.
- **Priority queue with lazy deletion:** multiple heap entries for the same state can exist; outdated ones are discarded when popped.

This design avoids costly decrease-key operations while preserving correctness.

3.2 Heuristics evaluated

Two heuristics are considered:

$h_0(s) = 0$: baseline (equivalent to Uniform Cost Search).

$h_1(s) = \# \text{empty}(s)$: number of remaining empty cells. This is admissible because each action fills exactly one cell, so at least $\# \text{empty}$ actions are required.

While h_1 is simple, it provides structured guidance: states closer to completion (fewer blanks) receive lower heuristic value, improving prioritization in the frontier.

3.3 Metrics

For each run we record:

- runtime (seconds),
- expanded nodes (pop/expand operations),
- generated nodes (successors produced),
- maximum frontier size (memory proxy),
- maximum closed size.

4 SAT Formulation

4.1 Variables

We introduce Boolean variables:

$$X_{r,c,v} \iff \text{cell } (r, c) \text{ contains digit } v, \quad r, c, v \in \{1, \dots, 9\}.$$

There are 729 variables.

4.2 CNF constraints

Sudoku is encoded with *exactly-one* constraints:

- **Cell:** each cell takes exactly one value;
- **Row:** each digit appears exactly once per row;
- **Column:** each digit appears exactly once per column;
- **Box:** each digit appears exactly once per 3×3 box;
- **Clues:** each given digit becomes a unit clause.

Each exactly-one is implemented as (i) one *at-least-one* clause and (ii) pairwise *at-most-one* clauses. The resulting CNF is then solved with a PySAT backend (e.g., Glucose), and the model is decoded into a completed grid and validated.

5 Experimental Setup

5.1 Benchmarks

We use Norvig’s benchmark sets <https://norvig.com/CQ/>:

- `easy50.txt`
- `top95.txt`
- `hardest.txt`

Each file contains one puzzle per line, represented by 81 characters (digits or . for empty).

5.2 Protocol

For each puzzle, we run:

- A* with h_0 ,
- A* with h_1 ,
- SAT solving.

Results are written to a CSV and then plotted automatically. For summary plots we report mean and standard deviation; because Sudoku difficulty within a benchmark can be heterogeneous, we also use distribution plots (boxplots) to avoid misinterpretation due to outliers.

6 Results and Discussion

6.1 Why these plots

The objective of the plots is to attribute performance to concrete causes:

- **Runtime distribution (boxplots)**: captures heterogeneity and outliers that $\text{mean} \pm \text{std}$ can hide.
- **Runtime ratio (A*/SAT)**: compares algorithms on a common scale even when SAT runtimes are near zero.
- **Runtime vs expanded (scatter)**: links time to search effort and validates that runtime is dominated by the number of expansions.

These plots together support a causal narrative rather than a purely descriptive one.

6.2 Runtime distributions

Figure 1 shows runtime distributions on `top95`. A* exhibits a heavy-tailed distribution: many instances are solved quickly, but a small subset generates large runtimes. SAT remains tightly concentrated close to zero, indicating stable performance.

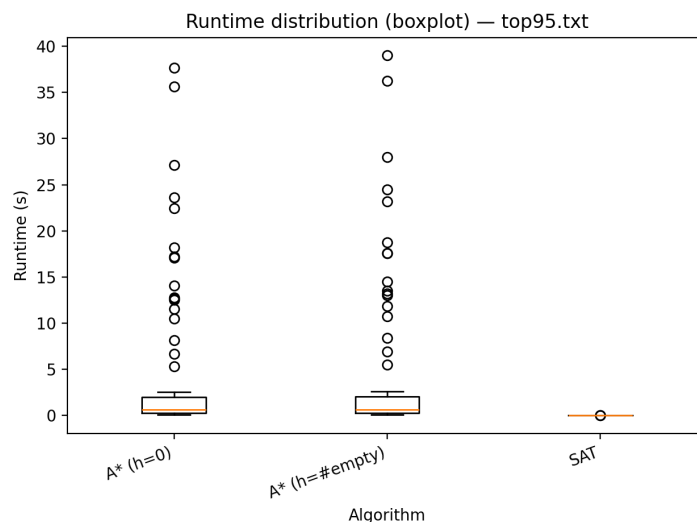


Figure 1: Runtime distribution on `top95` (boxplot). A* has a heavy-tailed distribution with multiple outliers, reflecting heterogeneous instance difficulty. SAT is stable with very low variance.

For completeness, Figure 2 reports **easy50** and **hardest**. On **easy50**, both A^* variants run in milliseconds with small dispersion; on **hardest**, A^* shows higher central tendency than **easy50** but fewer extreme outliers than **top95**, suggesting that “hardness” in the benchmark name does not necessarily coincide with worst-case search behavior.

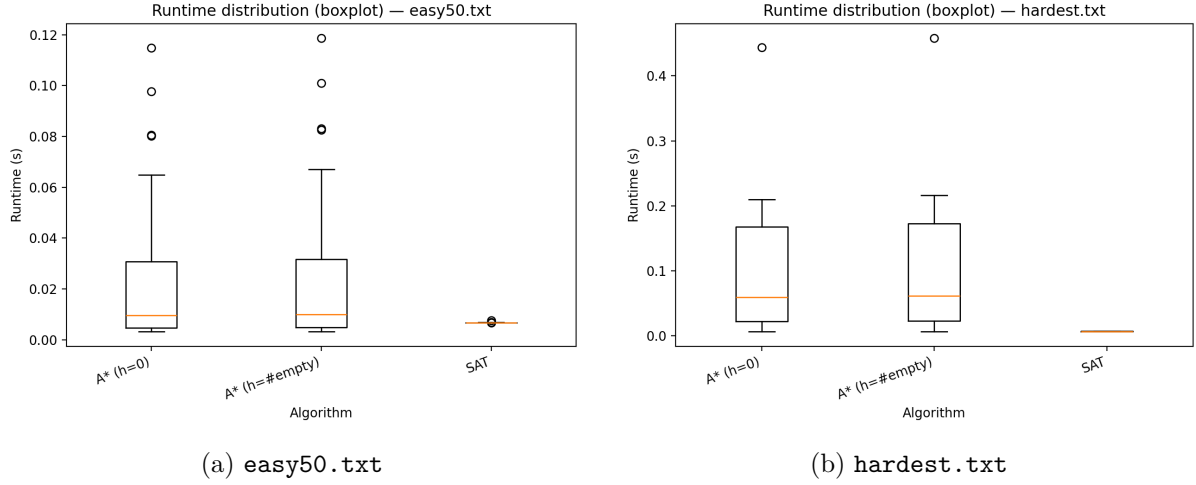


Figure 2: Runtime distributions (boxplots) on **easy50** and **hardest**. A^* is fast on **easy50**; on **hardest** A^* remains slower than SAT but shows fewer extreme outliers compared to **top95**.

6.3 Relative advantage of SAT

Figure 3 reports the distribution of runtime ratios (A^*/SAT) per benchmark. This representation avoids a compressed scale effect in which SAT would appear as a flat line near zero. The ratio plot makes the advantage of SAT explicit: on **top95**, A^* can be orders of magnitude slower for outlier instances; on **easy50** and **hardest**, the advantage is smaller but still systematic.

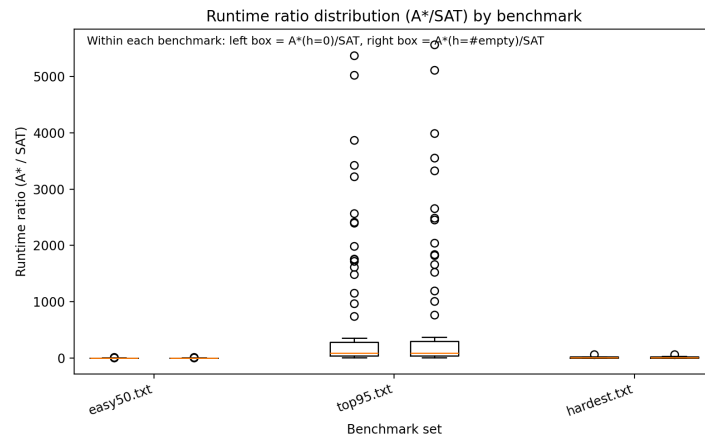


Figure 3: Runtime ratio distributions (A^*/SAT) by benchmark. Ratios provide a common scale for comparison when SAT runtimes are near zero. Outliers on **top95** indicate that a few instances dominate the search cost for A^* .

6.4 Runtime is dominated by search effort (A^*)

Figure 4 shows runtime versus expanded nodes for A^* with h_0 and h_1 . The near-linear trend indicates that runtime is primarily explained by the number of expanded states. This supports

the engineering interpretation that performance variability is caused by search tree size rather than incidental implementation overhead. The two heuristics produce similar trends; h_1 tends to reduce expansions for many instances, but worst-case instances still produce large search trees.

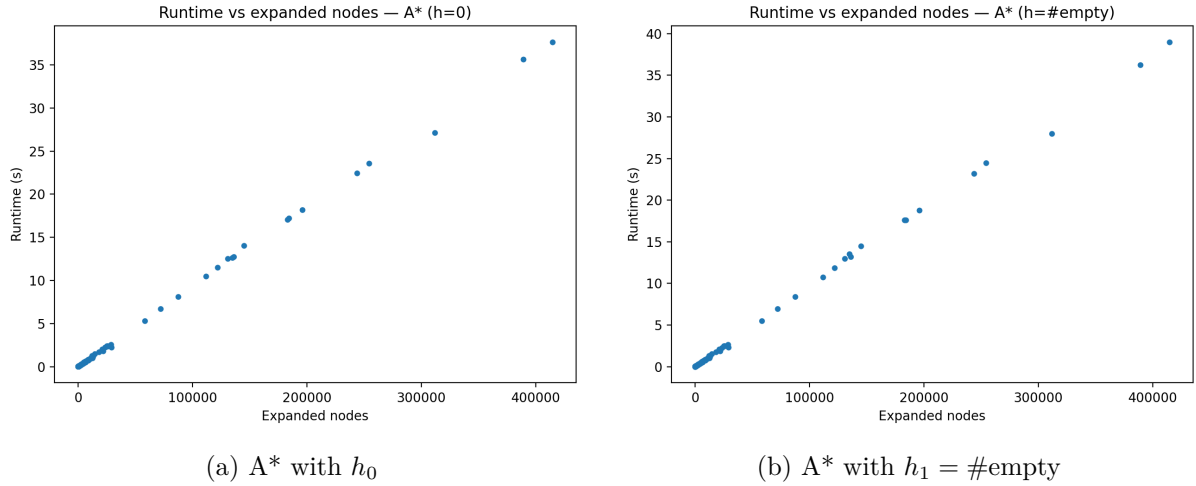


Figure 4: Runtime vs expanded nodes for A*. Runtime increases almost monotonically with expansions, indicating that search effort is the primary driver of runtime.

6.5 Summary of findings

The experimental evidence supports the following conclusions:

- **A* is competitive on easy instances:** MRV successor generation plus a simple admissible heuristic leads to millisecond-scale runtimes on `easy50`.
- **Instance heterogeneity matters:** `top95` contains a mixture of instances, some of which induce very large search trees, resulting in heavy-tailed runtime distributions and large outliers.
- **SAT is robust:** the SAT encoding remains structurally constant across instances (only clues change), and modern SAT solvers exploit this structure effectively, producing low runtime and low variance across benchmarks.
- **Heuristic impact:** h_1 generally reduces expansions relative to h_0 , but it does not eliminate worst-case behavior on certain instances; stronger heuristics or deeper constraint propagation would be needed for that.

7 Implementation Notes and How to Reproduce

Repository structure.

- `sudoku/core.py`: parsing, state representation, constraint checking, MRV successor generation
- `sudoku/astar.py`: A* implementation
- `sudoku/sat.py`: CNF encoding and SAT solving
- `scripts/run_experiments.py`: generates `results/results.csv`
- `scripts/plot_results.py`: generates plots in `results/plots/`

Commands.

```
[language=bash]
# Run experiments
python -m scripts.run_experiments \
  --benchmarks benchmarks/easy50.txt benchmarks/top95.txt benchmarks/hardest.txt \
  --out results/results.csv

# Generate plots
python -m scripts.plot_results --csv results/results.csv
```

8 Conclusion

This project compared A* search and SAT-based solving for Sudoku. A* provides an interpretable search process and performs well on many instances when combined with MRV and an admissible heuristic, but its runtime can vary dramatically due to heterogeneous instance difficulty, particularly in `top95`. SAT solving, based on a principled CNF encoding and a modern solver, delivers consistently fast and stable performance across all considered benchmarks. From an engineering standpoint, SAT is the most robust choice for this domain, while A* remains useful for analyzing search behavior and for cases where an explicit search trace or domain-specific heuristics are of interest.